filters) by height by width. We can turn this into a set of nodes by taking slices along the channel dimension where each node is then a channel-length vector, for a total of height times width number of nodes. We package these into a new matrix of dimension N × C, where N is the number of nodes and C is the channel dimension.

Figure 10.13. A high-level view of a self-attention-based relational module. The input to a relational module is a node matrix N: ..n×f with n nodes each with an f-dimensional feature vector. The relational module then copies this matrix for a total of three copies, and projects each one into a new matrix via a simple linear layer without an activation function, creating separate query, key, and value matrices. The query and key matrices are input to a compatibility function, which is any function that computes how compatible (similar in some way) each node is to each other node, resulting in a set of unnormalized attention weights, . This matrix is then normalized via the softmax function across the rows, such that each row's values will sum to 1. The value matrix and normalized attention matrix are then multiplied, . The output of the relational module is then usually passed through one or more linear layers (not depicted).

Figure 10.14. Left: Original MNIST digit for the number "5". Right: Transformed version that is translated to the top right and with random noise sprinkled in.

Figure 10.15. The projection step in self-attention. The input nodes are projected into a (usually) higher-dimensional feature space by simple matrix multiplication.

Figure 10.16. The loss and accuracy over training epochs for the relational module on classifying MNIST digits

Figure 10.17. Left column: Original input MNIST images (after transformation). Right column: Corresponding self-attention weights showing where the model is paying the most attention.

Figure 10.18. Multi-head dot product attention (MHDPA). Rather than use a single attention matrix, we can have multiple attention matrices called heads that can independently attend to different aspects of an input. The only difference is adding a new head dimension to the query, key and value tensors.